



PROJETO EM DUPLA — SISTEMA DE EVENTO GEEK



O desafio

Vocês foram contratados para desenvolver o sistema de gerenciamento de um grande **Evento Geek**, que terá competições, comidas, jogos e atividades para os participantes.

O sistema deverá permitir cadastrar participantes, registrar atividades, controlar pontuações e gerar um relatório final.

O projeto deverá ser desenvolvido **em dupla**.

A ideia é que os dois alunos trabalhem juntos, dividindo as partes do problema e depois integrando tudo em um único programa.



PARTE 1 — Cadastro dos participantes

O evento poderá receber até **5 participantes**.

Para cada participante, o programa deverá armazenar:

- Nome
- E-mail
- Telefone
- CEP
- Altura

Essas informações deverão ser armazenadas em uma **matriz 5×5**.

A estrutura deverá seguir esta organização:

	0	1	2	3	4
	Nome	E-mail	Telefone	CEP	Altura
Pessoa 0					
Pessoa 1					
Pessoa 2					
Pessoa 3					
Pessoa 4					

Por exemplo:

```
participantes = [  
    ["João", "joao@email.com", "99999-1111", "30000-000", 1.75],  
    ["Maria", "maria@email.com", "99999-2222", "30100-000", 1.62],  
]
```

```
    ...  
]
```

O programa deverá permitir cadastrar os participantes utilizando `input()`.

Ao final do cadastro:

Mostre todos os participantes cadastrados.

Exemplo:

```
===== PARTICIPANTES =====  
  
1 - João  
   E-mail: joao@email.com  
   Telefone: 99999-1111  
   CEP: 30000-000  
   Altura: 1.75  
  
2 - Maria  
   E-mail: maria@email.com  
   Telefone: 99999-2222  
   CEP: 30100-000  
   Altura: 1.62
```



PARTE 2 — Menu principal

Depois do cadastro, o programa deverá apresentar um menu:

```
=====
EVENTO GEEK 2026
=====

1 - Cadastrar participantes
2 - Ver participantes
3 - Registrar atividade
4 - Comprar produtos
5 - Ver pontuações
6 - Relatório final
0 - Encerrar programa

Escolha uma opção:
```

O menu deverá continuar aparecendo até que o usuário escolha `0`.

Utilize uma estrutura de repetição para controlar o menu.



PARTE 3 — Atividades do evento

O evento possui 5 atividades:

- 1 - Quiz de Programação
- 2 - Campeonato de Mario Kart
- 3 - Desafio de Minecraft
- 4 - Torneio de Pokémon
- 5 - Competição de Just Dance

Cada participante poderá participar das atividades.

Quando uma atividade for escolhida, o programa deverá perguntar:

Digite o número do participante:
Digite a pontuação obtida:

As pontuações deverão ser armazenadas.

Utilize uma **matriz para controlar as pontuações**.

Exemplo:

	Ativ. 1	Ativ. 2	Ativ. 3	Ativ. 4	Ativ. 5
João	80	90	0	70	0
Maria	95	0	85	90	100
Carlos	70	80	75	0	60

O 0 poderá representar que o participante ainda não participou daquela atividade.



PARTE 4 — Lanchonete do evento

Durante o evento existe uma lanchonete.

O cardápio possui:

- 1 - Hambúrguer R\$ 15,00
- 2 - Cachorro-quente ... R\$ 10,00
- 3 - Batata R\$ 8,00

```
4 - Refrigerante ..... R$ 6,00
5 - Milk-shake ..... R$ 12,00
```

O programa deverá permitir que um participante faça um pedido.

Pergunte:

```
Digite o número do participante:
```

Depois mostre o cardápio.

O participante poderá adicionar vários produtos ao pedido.

O pedido deverá terminar quando ele digitar:

```
0 - Finalizar pedido
```

O programa deverá:

- Registrar os produtos;
- Contabilizar a quantidade;
- Calcular o total;
- Mostrar o resumo da compra.

Exemplo:

```
===== PEDIDO =====

Hambúrguer: 2
Cachorro-quente: 0
Batata: 1
Refrigerante: 2
Milk-shake: 0

Total de itens: 5
Total: R$ 50,00
```



PARTE 5 — Funções

O programa deverá utilizar funções para organizar o código.

Criem pelo menos as seguintes funções:

```
mostrar_menu()
```

Responsável por exibir o menu principal.

```
cadastrar_participante()
```

Responsável pelo cadastro.

```
mostrar_participantes()
```

Responsável por exibir os participantes.

```
registrar_atividade()
```

Responsável por registrar uma pontuação.

```
calcular_media()
```

Recebe as pontuações de um participante e calcula sua média.

```
fazer_pedido()
```

Responsável pelo sistema da lanchonete.

```
relatorio_final()
```

Responsável por apresentar os resultados do evento.



PARTE 6 — Relatório individual

O usuário deverá conseguir consultar um participante.

O programa deverá pedir:

Digite o número do participante:

E mostrar:

===== FICHA DO PARTICIPANTE =====

Nome: João
E-mail: joao@email.com
Telefone: 99999-1111
CEP: 30000-000
Altura: 1.75

----- ATIVIDADES -----

Quiz de Programação: 80
Mario Kart: 90
Minecraft: 75
Pokémon: 70
Just Dance: 85

Média: 80

----- COMPRAS -----

Total gasto: R\$ 65,00



PARTE 7 — Ranking

Agora começa a parte mais importante.

O sistema deverá calcular a pontuação total de cada participante.

Por exemplo:

João: 400 pontos
Maria: 450 pontos
Carlos: 380 pontos
Ana: 420 pontos
Pedro: 350 pontos

Depois, mostre o ranking.

===== RANKING =====

1º - Maria - 450 pontos
2º - Ana - 420 pontos
3º - João - 400 pontos
4º - Carlos - 380 pontos
5º - Pedro - 350 pontos

O ranking deverá ser calculado pelo programa.



PARTE 8 — Estatísticas do evento

O relatório final também deverá apresentar algumas estatísticas.

Descubram:

Participantes

- Quantidade total de participantes;
- Participante mais alto;
- Participante mais baixo;
- Média de altura.

Atividades

- Maior pontuação registrada;
- Menor pontuação registrada;
- Média geral das pontuações;
- Atividade que recebeu mais participantes.

Lanchonete

- Quantidade total de produtos vendidos;
- Produto mais vendido;
- Valor total arrecadado.



PARTE 9 — Validações

O sistema não pode simplesmente aceitar qualquer coisa.

Por exemplo:

Se o usuário digitar:

9

para escolher uma atividade, deverá aparecer:

Opção inválida!

Se tentar cadastrar uma pontuação negativa:

Pontuação inválida!
Digite um valor entre 0 e 100.

Se escolher um participante inexistente:

Participante não encontrado!

O programa deverá continuar funcionando normalmente.



PARTE 10 — Organização do projeto

O código deverá ser organizado de maneira semelhante a:

```
EVENTO GEEK
├── Variáveis e listas
├── Funções
│   ├── mostrar_menu()
│   ├── cadastrar_participante()
│   ├── mostrar_participantes()
│   ├── registrar_atividade()
│   ├── calcular_media()
│   ├── fazer_pedido()
│   └── relatorio_final()
└── Programa principal
    └── Menu
```

Evitem colocar todo o código dentro de um único bloco enorme.



TRABALHO EM DUPLA

A dupla deverá dividir o desenvolvimento.

Aluno 1

Pode ficar responsável inicialmente por:

- Cadastro;
- Matriz dos participantes;
- Exibição dos participantes;
- Menu.

Aluno 2

Pode ficar responsável inicialmente por:

- Atividades;
- Pontuações;
- Lanchonete;
- Cálculos.

Depois, os dois deverão juntar as partes e fazer a integração.

Importante: os dois alunos precisam entender o código inteiro. Não vale simplesmente copiar a parte do colega sem saber explicar como ela funciona.

DESAFIOS EXTRAS

Se terminarem antes do restante da turma, implementem os desafios abaixo.

Desafio 1 — Buscar participante

Adicione uma opção:

```
7 - Buscar participante
```

O usuário poderá digitar o nome e o programa deverá procurar na matriz.

Desafio 2 — Participante VIP

Se um participante gastar mais de **R\$ 100,00** na lanchonete, ele recebe o status:

```
 PARTICIPANTE VIP
```

Desafio 3 — Medalhas

Crie uma classificação:

100 pontos ou mais → 🥇 Ouro
70 a 99 → 🥈 Prata
40 a 69 → 🥉 Bronze
Abaixo de 40 → Participante

★ Desafio 4 — Menu completo

Adicione uma opção:

8 - Limpar pontuações

O sistema deverá permitir que o organizador zere a pontuação de um participante.



DESAFIO FINAL DA DUPLA

Imaginem que amanhã haverá outro evento.

O código deverá funcionar novamente **sem precisar ser reescrito**.

Portanto, evitem colocar informações fixas desnecessariamente.

O programa deverá ser capaz de:

```
Cadastrar
  ↓
Consultar
  ↓
Registrar atividades
  ↓
Registrar compras
  ↓
Calcular estatísticas
  ↓
Gerar ranking
  ↓
Gerar relatório final
```



Objetivo

Ao final do projeto, vocês deverão ter construído um pequeno **sistema completo em Python**, utilizando os conteúdos estudados até agora:

- `input()`

- `print()`
- Variáveis
- Números
- Operações matemáticas
- `if / elif / else`
- `for`
- `while`
- Listas
- Matrizes
- Funções
- Parâmetros e retornos
- Contadores
- Acumuladores
- Validação de dados
- Organização de código

Não existe uma única maneira correta de resolver o projeto. A dupla deverá decidir como organizar as listas, matrizes, funções e regras do sistema.